

This handout includes space for every question that requires a written response. Please feel free to use it to handwrite your solutions (legibly, please). If you choose to typeset your solutions, the `README.md` for this assignment includes instructions to regenerate this handout with your typeset L^AT_EX solutions.

1.a

1.b **Answer:** $O(mnp)$.

Justification. The product $C = AB$ has $m \cdot p$ entries, and each entry is an inner product of length n :

$$C_{ij} = \sum_{k=1}^n A_{ik} B_{kj},$$

so each entry costs n multiply–adds. The total is therefore $mp \cdot n = O(mnp)$.

Two caveats. This is the *standard* (naive) algorithm — Strassen-style methods beat $O(n^3)$ on square matrices. And as a sanity check, the square case $m = n = p$ gives $O(n^3)$, as expected.

1.c

1.d (i) Xw , **shapes** $(n, d), (d,) \rightarrow (n,)$

```
einsum(X, w, 'n d, d -> n')
```

The single axis of w is labelled d so that it aligns with the column axis of X . Since d is absent from the output it is summed over, giving $(Xw)_i = \sum_d X_{id} w_d$. The axis n is carried through, so the result has one entry per row of X .

(ii) $XX^\top$, **shapes** $(n, d), (n, d) \rightarrow (n, n)$

```
einsum(X, X, 'i d, j d -> i j')
```

Entry $[i, j]$ is the dot product of row i with row j , that is $(XX^\top)_{ij} = \sum_d X_{id} X_{jd}$. The two row axes index *different* rows, so they must receive different letters i and j , both of which are kept in the output. The feature axis d is shared by both operands and dropped from the output, so it is contracted. No explicit transpose is required — the transpose is expressed by *which letters line up*, not by flipping an array.

(iii) $\text{diag}(X^\top X)$, **shapes** $(n, d), (n, d) \rightarrow (d,)$

```
einsum(X, X, 'n d, n d -> d')
```

The k -th diagonal entry of $X^\top X$ is the squared norm of column k : $\text{diag}(X^\top X)_d = \sum_n X_{nd} X_{nd} = \sum_n X_{nd}^2$. Both operands are the plain array X , so both are labelled $n d$. The row axis n is absent from the output and so is summed; the feature axis d appears in both inputs *and* the output, so it is kept and aligned elementwise. The net effect is to square X elementwise and then sum down the rows.

Note that $X^\top X$ is itself $d \times d$; it is `diag` that collapses it to a length- d vector. Computing it as `np.diag(X.T @ X)` would build the full $d \times d$ matrix only to discard the off-diagonal entries — $O(nd^2)$ work for d useful numbers — whereas the `einsum` string never forms that matrix and costs $O(nd)$. The cost can be read straight off the letters, as in 1b: `'n d, n d -> d'` has only two distinct letters, hence two loops.

2.a
2.c

3.a

3.b

4.a

4.b

4.c

4.d

5.a

5.b

5.c

5.d